

9.1 Spring - Transaction Management

A database transaction is a sequence of actions that are treated as a single unit of work. These actions should either complete entirely or take no effect at all. Transaction management is an important part of RDBMS-oriented enterprise application to ensure data integrity and consistency.

9.2 Securing REST with Spring Security

Spring Security is an application-level security framework which provides various security features like: authentication, authorization to create secure Java Enterprise Applications. To enable application-level security feature, add spring security dependency to the pom.xml or you can add while creating a maven spring boot application using spring initilizr. Spring security make sure that each and every API written in the controller to be authenticated.

The framework targets two major areas of application are authentication and authorization.

Authentication is the process of knowing and identifying the user that wants to access.

Authorization is the process to allow authority to perform actions in the application. The application-level framework features provide login and logout functionality. It allow/block access to URLs to logged in users. It also allow/block access to URLs to logged in users AND with certain roles.

po.xml code for spring security dependency

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

SecurityController.java class

```
@RestController
public class SecurityController {

    @GetMapping("/")
    public String Welcome() {
        return ("<h1>Welcome to SpringBoot Security</h1>");
    }
}
```

www.INSTELLO.in

A default system generated security password is given by the spring boot security framework. The password is dynamic and changes every time we execute the application. Try to access the REST API using the URI mapping that is `http://localhost:8080/` in any browser or application like postman or swagger. As the URI hits in the browser a default login form authentication is enabled. By default, User is created as username for authentication. Spring security also provide logout form and error handling / validation.

We can make the password static by setting its value in the `application.properties` file. `Application.properties` file is created under `src/main/resource` package. We can also create static user and static password by setting its value in the `application.properties` file as shown below.

```
spring.security.user.name=ABC
spring.security.user.password=XYZ
```

9.3 API security

Application programming interface (API) security refers to the practice of preventing or mitigating attacks on APIs. APIs work as the backend framework for mobile and web applications. Therefore, it is critical to protect the sensitive data they transfer. An API is an interface that defines how different software interacts. It controls the types of requests that occur between programs, how these requests are made, and the kinds of data formats that are used. APIs are used in Internet of Things (IoT) applications and on websites.

They often gather and process data or allow the user to input information that gets processed within the environment housing the API. For example, there is an API that runs Google Maps. A web designer can embed Google Maps into a page they are building. When the user uses Google Maps, they are not using code the web designer wrote piece by piece, but they are simply using a prewritten API provided by Google. API security covers the APIs you own, as well as the ones you use indirectly.

9.4 What is user authentication flow?

The purpose of the Authentication flow is to identify and authenticate the user to the Payment Integrator (Integrator). Authentication is an input to other methods. Particularly for associateAccount and capture. This means

that the proof of authentication is used as an input (parameter) to those two methods. The diagram shows flow of how we implement User Registration, User Login and Authorization process.

Spring Boot and Authorization

Authorization functionality is provided by Spring Security and allows the user to be authorized before accessing resources. It enables developers to set access controls for resources. Recall the Identity and Access Management (IAM) flow:

1. Users will start by authenticating with a username and password managed by Auth0.
2. Once authenticated, the client will receive a JWT representing an access token.
3. The client will include the access token in the authorization header of every request to a secure endpoint
4. The server will validate the access token and determine if it has the right permissions, using the information within the token.

9.5 JUnit

JUnit is a unit testing open-source framework for the Java programming language. Java Developers use this framework to write and execute automated tests. In Java, there are test cases that have to be re-executed every time a new code is added. This is done to make sure that nothing in the code is broken.

JUnit has several graphs that represent the progress of a test. When the test runs smoothly, the graph displays a green color, and it turns red if the test fails. JUnit Testing enables developers to develop highly reliable and bug-free code.

What is the need for JUnit Testing?

The top reasons to take up JUnit Testing are:

- To find bugs early in the development phase, which increases the code's reliability
- The framework enables the developer to invest more time in reading the code than writing it
- This makes the code more readable, reliable, and bug-free

- It boosts the confidence of the developer and motivates them immensely

9.6 Test Controller endpoints

You will build a simple Spring application and test it with JUnit. You probably already know how to write and run unit tests of the individual classes in your application, so, for this guide, we will concentrate on using Spring Test and Spring Boot features to test the interactions between Spring and your code. You will start with a simple test that the application context loads successfully and continue on to test only the web layer by using Spring's MockMvc.

9.7 Introduction to NoSQL

NoSQL originally referring to non SQL or non relational is a database that provides a mechanism for storage and retrieval of data. This data is modeled in means other than the tabular relations used in relational databases.

NoSQL database technology stores information in JSON documents instead of columns and rows used by relational databases. To be clear, NoSQL stands for "not only SQL" rather than "no SQL" at all. This means a NoSQL JSON database can store and retrieve data using literally "no SQL." Or you can combine the flexibility of JSON with the power of SQL for the best of both worlds. Consequently, NoSQL databases are built to be flexible, scalable, and capable of rapidly responding to the data management demands of modern businesses.

9.7.1 Brief History

NoSQL databases emerged in the late 2000s as the cost of storage dramatically decreased. Gone were the days of needing to create a complex, difficult-to-manage data model in order to avoid data duplication. Developers (rather than storage) were becoming the primary cost of software development, so NoSQL databases optimized for developer productivity.

9.7.2 Features and Benefits

Features of NoSQL are:

1. it never follows relational database model.
2. it never provides table with flat fixed column record.
3. it is schema free.
4. it provides share nothing environment.
5. it is scalable.
6. it has low-cost hardware.
7. it provides faster performance.
8. Example : MongoDB, Cassandra.

Benefits of NoSQL: There are many advantages of working with NoSQL databases such as MongoDB and Cassandra. The main advantages are high scalability and high availability.

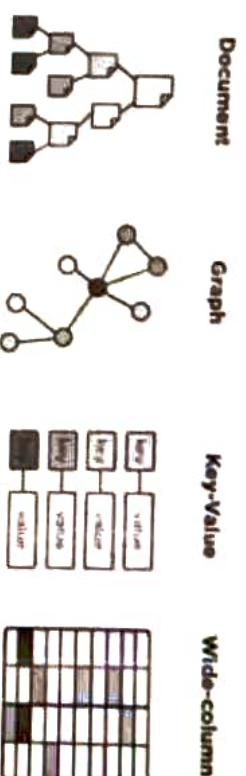
1. High scalability – NoSQL databases use sharding for horizontal scaling. Partitioning of data and placing it on multiple machines in such a way that the order of the data is preserved is sharding. Vertical scaling means adding more resources to the existing machine whereas horizontal scaling means adding more machines to handle the data. Vertical scaling is not that easy to implement but horizontal scaling is easy to implement. Examples of horizontal scaling databases are MongoDB, Cassandra, etc..

2. High availability – Auto replication feature in NoSQL databases makes it highly available because in case of any failure data replicates itself to the previous consistent state.

9.7.3 Types of NoSQL database

Types of NoSQL databases and the name of the databases system that falls in that category are:

1. **Graph Databases:** Amazon Neptune, Neo4j
2. **Key value store:** Memcached, Redis, Coherence
3. **Column-oriented:** Apache Cassandra, DataStax, Microsoft Azure Cosmos DB, and ScyllaDB
4. **Document-based:** MongoDB, CouchDB, Cloudbant



Document databases: A document database stores data in JSON, BSON, or XML documents (not Word documents or Google Docs, of course). In a document database, documents can be nested. Particular elements can be indexed for faster querying. Documents can be stored and retrieved in a form that is much closer to the data objects used in applications, which means less translation is required to use the data in an application. SQL data must often be assembled and disassembled when moving back and forth between applications and storage.

Key-value stores: The simplest type of NoSQL database is a key-value store. Every data element in the database is stored as a key value pair consisting of an attribute name (or "key") and a value. In a sense, a key-value store is like a relational database with only two columns: the key or attribute name (such as "state") and the value (such as "Alaska"). Use cases include shopping carts, user preferences, and user profiles.

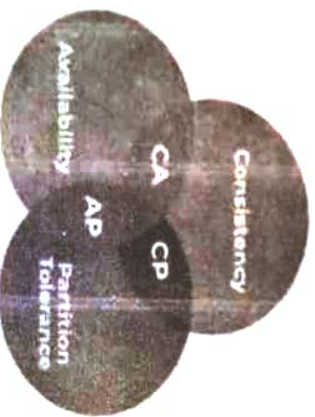
Column-oriented databases: While a relational database stores data in rows and reads data row by row, a column store is organized as a set of columns. This means that when you want to run analytics on a small number of columns, you can read those columns directly without consuming memory with the unwanted data. Columns are often of the same type and benefit from more efficient compression, making reads even faster. Columnar databases can quickly aggregate the value of a given column (adding up the total sales for the year, for example). Use cases include analytics.

Graph databases A graph database focuses on the relationship between data elements. Each element is stored as a node (such as a person in a social media graph). The connections between elements are called links or relationships. In a graph database, connections are first-class elements of the database, stored directly. In relational databases, links are implied, using data to express the relationships.

9.7.4 The CAP Theorem in DBMS

The CAP theorem, originally introduced as the CAP principle, can be used to explain some of the competing requirements in a distributed system with replication. It is a tool used to make system designers aware of the trade-offs while designing networked shared-data systems.

The three letters in CAP refer to three desirable properties of distributed systems with replicated data: **consistency** (among replicated copies), **availability** (of the system for read and write operations) and **partition tolerance** (in the face of the nodes in the system being partitioned by a network fault).



The theorem states that networked shared-data systems can only strongly support two of the following three properties:

Consistency

Consistency means that the nodes will have the same copies of a replicated data item visible for various transactions. A guarantee that every node in a distributed cluster returns the same, most recent and a successful write. Consistency refers to every client having the same view

of the data. There are various types of consistency models. Consistency in CAP refers to sequential consistency, a very strong form of consistency.

Availability

Availability means that each read or write request for a data item will either be processed successfully or will receive a message that the operation cannot be completed. Every non-failing node returns a response for all the read and write requests in a reasonable amount of time. The key word here is "every". In simple terms, every node (on either side of a network partition) must be able to respond in a reasonable amount of time.

Partition Tolerance

Partition tolerance means that the system can continue operating even if the network connecting the nodes has a fault that results in two or more partitions, where the nodes in each partition can only communicate among each other. That means, the system continues to function and upholds its consistency guarantees in spite of network partitions. Network partitions are a fact of life. Distributed systems guaranteeing partition tolerance can gracefully recover from partitions once the partition heals.

9.7.5 BASE Model For Database

The rise in popularity of NoSQL databases provided a flexible and fluidity with ease to manipulate data and as a result, a new database model was designed, reflecting these properties. The acronym BASE is slightly more confusing than ACID but however, the words behind it suggest ways in which the BASE model is different and acronym BASE stands for:-

Basically Available: Instead of making it compulsory for immediate consistency, BASE-modelled NoSQL databases will ensure the availability of data by spreading and replicating it across the nodes of the database cluster.

Soft State: Due to the lack of immediate consistency, the data values may change over time. The BASE model breaks off with the concept of a database that obligates its own consistency, delegating that responsibility to developers.

Eventually Consistent: The fact that BASE does not obligates immediate consistency but it does not mean that it never achieves it. However, until it does, the data reads are still possible (even though they might not reflect reality).

9.8 MongoDB

MongoDB is an open-source document-oriented database that is designed to store a large scale of data and also allows you to work with that data very efficiently. It is categorized under the NoSQL (Not only SQL) database because the storage and retrieval of data in the MongoDB are not in the form of tables.

9.8.1 Features of MongoDB

Schema-less Database:

It is the great feature provided by the MongoDB. A Schema-less database means one collection can hold different types of documents in it. Or in other words, in the MongoDB database, a single collection can hold multiple documents and these documents may consist of the different numbers of fields, content, and size. It is not necessary that the one document is similar to another document like in the relational databases. Due to this cool feature, MongoDB provides great flexibility to databases. **Document Oriented:** In MongoDB, all the data stored in the documents instead of tables like in RDBMS. In these documents, the data is stored in fields (key-value pair) instead of rows and columns which make the data much more flexible in comparison to RDBMS. And each document contains its unique object id.

Indexing: In MongoDB database, every field in the documents is indexed with primary and secondary indices this makes easier and takes less time to get or search data from the pool of the data. If the data is not indexed, then database search each document with the specified query which takes lots of time and not so efficient.

Scalability: MongoDB provides horizontal scalability with the help of sharding. Sharding means to distribute data on multiple servers, here a large amount of data is partitioned into data chunks using the shard key, and these data chunks are evenly distributed across shards that reside

across many physical servers. It will also add new machines to a running database.

Replication: MongoDB provides high availability and redundancy with the help of replication. It creates multiple copies of the data and sends these copies to a different server so that if one server fails, then the data is retrieved from another server.

Aggregation: It allows to perform operations on the grouped data and get a single result or computed result. It is similar to the SQL GROUPBY clause. It provides three different aggregations i.e, aggregation pipeline, map-reduce function, and single-purpose aggregation methods

High Performance: The performance of MongoDB is very high and data persistence as compared to another database due to its features like scalability, indexing, replication, etc.

How MongoDB is different from RDBMS ?

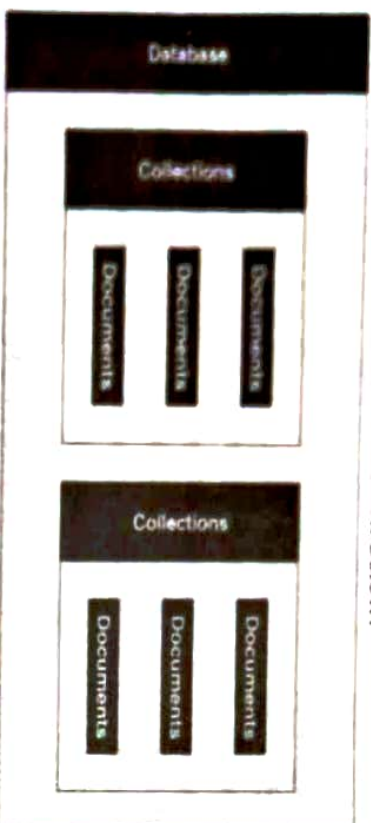
Some major differences in between MongoDB and the RDBMS are as follows:

MongoDB	RDBMS
It is a non- relational and document- oriented database.	It is a relational database.
It is suitable for hierarchical data storage.	It is not suitable for hierarchical data storage
It has a dynamic schema.	It has a predefined schema.
It centers around the CAP theorem (Consistency, Availability, and Partition tolerance).	It centers around ACID properties (Atomicity, Consistency, Isolation, and Durability).
In terms of performance, it is much faster than RDBMS	In terms of performance, it is slower than Mongo DB

9.8.2 The Architecture of MongoDB NoSQL Database

Now, we will see how actually thing happens behind the scene. As we know that MongoDB is a database server and the data is stored in these

databases. Or in other words, MongoDB environment gives you a server that you can start and then create multiple databases on it using MongoDB. Because of its NoSQL database, the data is stored in the collections and documents. Hence the database, collection, and documents are related to each other as shown below:



Database: In simple words, it can be called the physical container for data. Each of the databases has its own set of files on the file system with multiple databases existing on a single MongoDB server. The MongoDB database contains collections just like the MySQL database contains tables. You are allowed to create multiple databases and multiple collections.

Collection: A group of database documents can be called a collection. The RDBMS equivalent to a collection is a table. The entire collection exists within a single database. There are no schemas when it comes to collections. Inside the collection, various documents can have varied fields, but mostly the documents within a collection are meant for the same purpose or for serving the same end goal. Now inside of the collection we have documents. These documents contain the data we want to store in the MongoDB database and a single collection can contain multiple documents and you are schema-less means it is not necessary that one document is similar to another.

Document: A set of key-value pairs can be designated as a document. Documents are associated with dynamic schemas. The benefit of having dynamic schemas is that a document in a single collection does not have

to possess the same structure or fields. Also, the common fields in a collection document can have varied types of data.

The documents are created using the fields. Fields are key-value pairs in the documents, it is just like columns in the relation database. The value of the fields can be of any BSON data types like double, string, boolean, etc.

- The data stored in the MongoDB is in the format of BSON documents. Here, BSON stands for Binary representation of JSON documents. Or in other words, in the backend, the MongoDB server converts the JSON data into a binary form that is known as BSON and this BSON is stored and queried more efficiently.

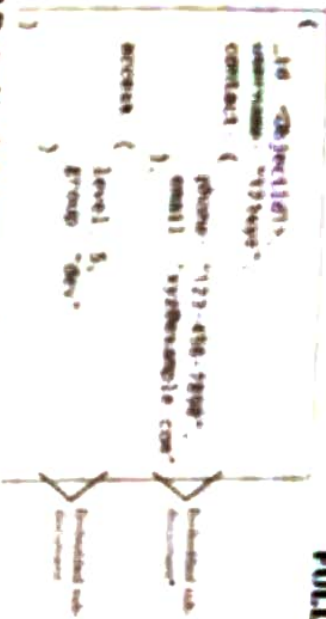
- In MongoDB documents, you are allowed to store nested data. This nesting of data allows you to create complex relations between data and store them in the same document which makes the working and fetching of data extremely efficient as compared to SQL. In SQL, you need to write complex joins to get the data from table 1 and table 2. The maximum size of the BSON document is 16MB.

9.8.3 Data Model Design

Data in MongoDB has a flexible schema. documents in the same collection. They do not need to have the same set of fields or structure. Common fields in a collection's documents may hold different types of data. MongoDB provides two types of data models: — Embedded data model and Normalized data model. Based on the requirement, you can use either of the models while preparing your document.

Embedded Data Model

In this model, you can have (embed) all the related data in a single document, it is also known as de-normalized data model. For example, assume we are getting the details of employees in three different documents namely, Personal_details, Contact and, Address, you can embed all the three documents in a single one as shown below -



Normalized Data Model: In general, use normalized data models:

- when embedding would result in duplication of data but would not provide sufficient read performance advantages to outweigh the implications of the duplication.
- to represent more complex many-to-many relationships.
- to model large hierarchical data sets.

In this model, you can refer the sub documents in the original document, using references. For example, you can re-write the above document in the normalized model as:

Employee:

```
{
  _id: <ObjectId101>;
  Emp_ID: "10025AE336";
}
```

Personal_details:

```
{
  _id: <ObjectId102>;
  empDocID: " ObjectId101";
  First_Name: "Radhika";
  Last_Name: "Sharma";
  Date_Of_Birth: "1995-09-26"
}
```

9.8.4 MongoDB Shell (mongosh)

The MongoDB Shell, mongosh, is a fully functional JavaScript and Node.js 16.x REPL environment for interacting with MongoDB deployments. You can use the MongoDB Shell to test queries and operations directly with your database.

www.INSTELLO.in

Install from MSI

1. Open the MongoDB Shell download page. Open the MongoDB Download Center.
2. In the Platform dropdown, select Windows 64-bit (8.1+) (MSI)
3. Click Download.
4. Double-click the installer file.
5. Follow the prompts to install mongosh.

9.8.5 MongoDB Atlas

MongoDB has a cloud service also known as MongoDB Atlas. It is a cloud base database that allows us to host up our database and serve us whenever we need it. It removes all our effort to host our database on our local devices and can serve us anytime. Features of MongoDB Atlas:

- Strong security
- More precise analysis of data
- Easy Scalability
- Technical support

9.8.6 MongoDB Compass

MongoDB Compass is a GUI for MongoDB. It is also known as MongoDB GUI. MongoDB allows users to analyze the content of their stored data without any prior knowledge of MongoDB query syntax. When we explore exploring our data in the visual environment, we can use Compass GUI to optimize performance, manage indexes, and implement document validation.

9.8.7 MongoDB - Datatypes

MongoDB supports many datatypes. Some of them are –

String – This is the most commonly used datatype to store the data. String in MongoDB must be UTF-8 valid.

Integer – This type is used to store a numerical value. Integer can be 32 bit or 64 bit depending upon your server.

Boolean – This type is used to store a boolean (true/false) value.

Double – This type is used to store floating point values.

Min/ Max keys – This type is used to compare a value against the low and highest BSON elements.

Arrays – This type is used to store arrays or list or multiple values into one key.

Timestamp – This can be handy for recording when a document has been modified or added.

Object – This datatype is used for embedded documents.

Null – This type is used to store a Null value.

Symbol – This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.

Date – This datatype is used to store the current date or time in UNIX time format. You can specify your own date time by creating object of Date and passing day, month, year into it.

Object ID – This datatype is used to store the document's ID.

Binary data – This datatype is used to store binary data.

9.8.8 Operators in Mongo DB

Query and Projection operators: Query operators provide ways to locate data within the database and projection operators modify how data is presented.

Update Operators: Update operators are operators that enable you to modify the data in your database or add additional data.

Aggregation Pipeline Operators: Aggregation pipeline operations have a collection of operators available to define and manipulate documents in pipeline stages.

API Gateway: An API gateway is an API management tool that sits between a client and a collection of backend services. An API gateway acts as a reverse proxy to accept all application programming interface (API) calls, aggregate the various services required to fulfill them, and return the appropriate result.

IMPORTANT QUESTIONS

1. Compare the databases MySQL and MongoDB. (DECEMBER 2023 / MAY 2024 / DECEMBER 2024, 10 MARKS)

MySQL	MongoDB
1. It is relational database	1. It is NoSQL database.
2. Suitable for Structured data with clear schema	2. Suitable for Unstructured data
3. Represents data as tables and rows	3. Represents data as JSON docs
4. Must specify tables and columns	4. No need to declare the schema
5. Allows Join operations	5. Doesn't allow Join operations
6. Utilizes Structured query language	6. Query language is Javascript
7. More suitable when you are just starting the database which doesn't scale much	7. More suitable when you need high availability Of data with auto data recovery
8. There is no auto failure recovery	8. It supports auto failure recovery
9. It is Slower	9. It is faster than MySQL
10. Works better for smaller datasets	10. Works better for larger datasets

2. Demonstrate with a simple code to secure REST APIs with Spring Security. (SEPTEMBER 2023 / JULY 2024, 10 MARKS) Refer Topic Number 9.2 in Page Number 165

WEEKLY ASSIGNMENT 9

1. Demonstrate with a simple code to secure REST APIs with Spring Security.
2. Compare the databases MySQL and MongoDB